

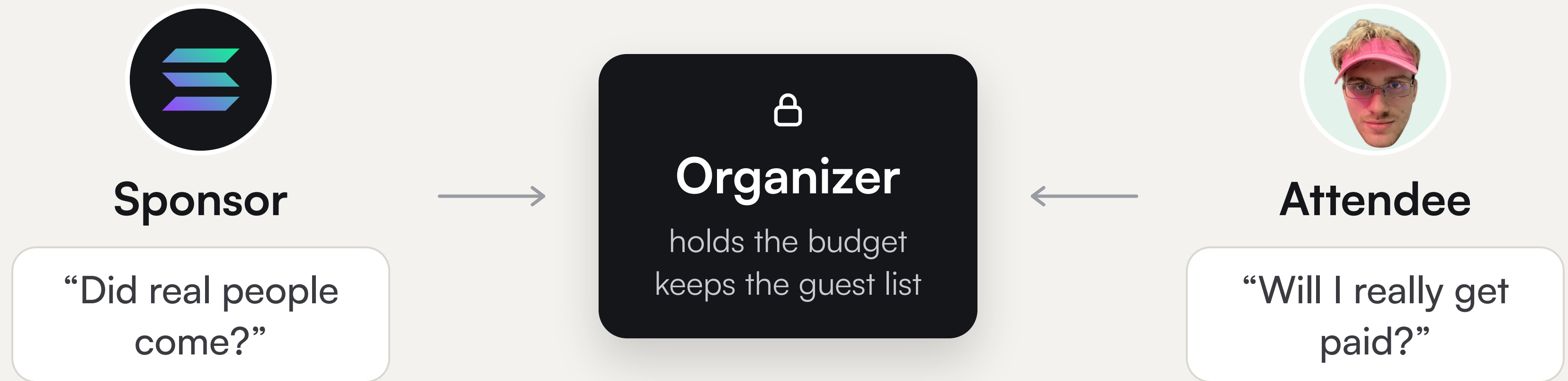


Seen on site, **paid on-chain.**

Attendance giveaways that pay out by themselves.

THE PROBLEM

Today's giveaways run on trust in a middleman.



If the middleman is wrong, slow or dishonest, **nobody outside can check.**

WHO IT'S FOR

Built for organizers who reward showing up.

FOR

Meetups

Hackathons

Product launches

Conference side events

ORGANIZERS & SPONSORS

 **Contract**



Set the reward

Lock the budget

Put a camera at the door

THEIR ATTENDEES

First 100 attendees earn

0.01 SOL

Sign up with MetaMask



 Powered by OnSight

WHAT CHANGES

The program does the middleman's job.

TODAY

✗ The organizer holds the budget

✗ Rules are a promise

✗ Staff decide who came

✗ Paid later, by hand, if at all

✗ Nobody can audit the list

WITH ONSIGHT

✓ Locked in escrow when the event is created

✓ Frozen on chain: reward, cap, window, oracles

✓ Oracles report, the program decides

✓ Paid in the same transaction, seconds later

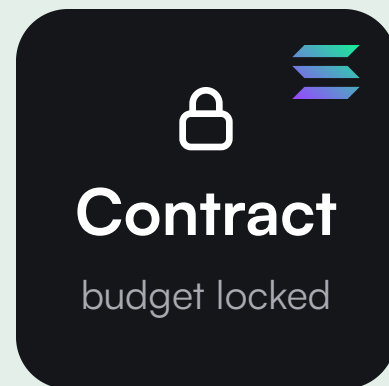
✓ Every payout is public on Solana Explorer

HOW IT WORKS

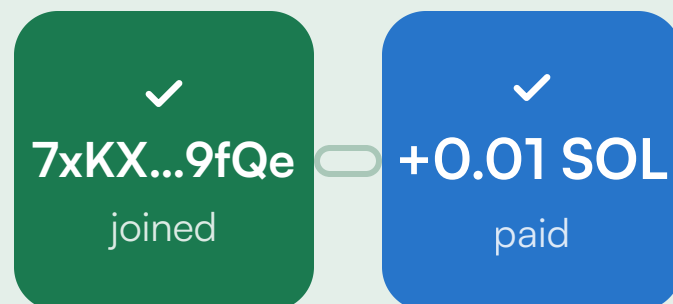
Oracles see faces. The program decides.

ON CHAIN • SOLANA

- 1 Organizer locks the budget
`create_event`

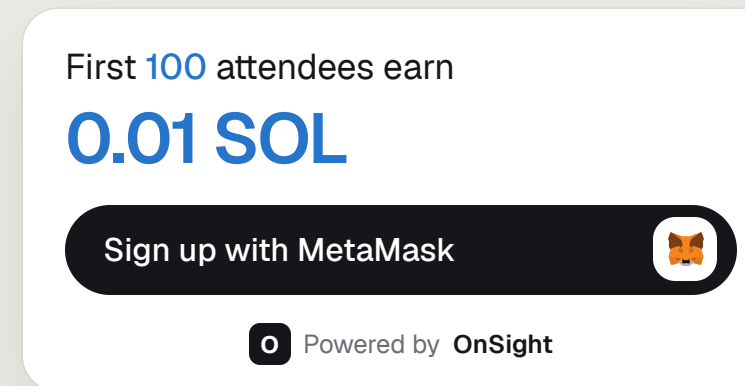


- 4 `report_sighting` pays
straight from escrow

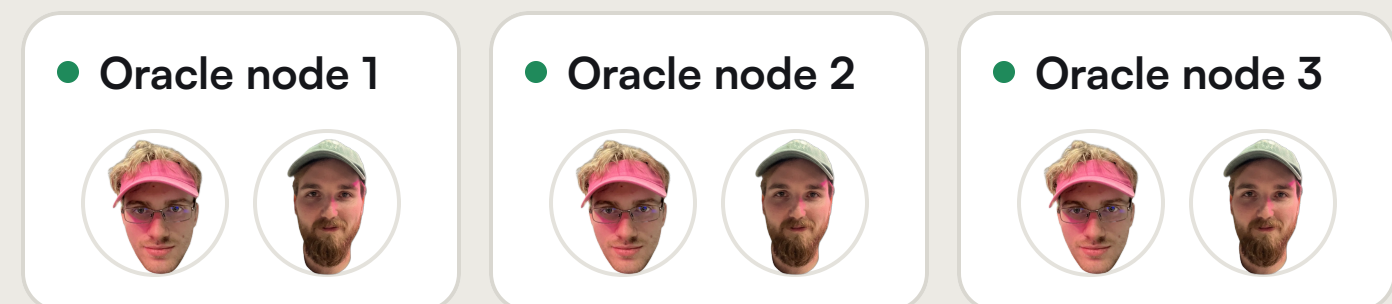


OFF CHAIN

- 2 Attendee joins: wallet,
selfie, signature



- 3 At the door, oracles report
"I see this wallet"



WHERE EXACTLY

The middleman disappears in `report_sighting`.

```
pub fn report_sighting(ctx) -> Result<()> {  
  ev.oracles.position(oracle).ok_or(NotOracle)?;  
  check_join_proof(event, attendee)?;  
  require!(now >= ev.start && now <= ev.end);  
  attendance.reporters |= 1 << slot;  
  if !(enough_oracles && long_enough) { return Ok(()) }  
  require!(ev.paid_count < ev.max_paid, CapReached);  
  attendance.paid = true;  
  event.sub_lamports(reward + fee)?;  
  attendee.add_lamports(reward)?;  
}
```

- 1 Only an oracle listed on the event
- 2 Only with the attendee's own signed join
- 3 Only during the event, on the chain's clock
- 4 Only once M-of-N oracles saw them long enough
- 5 Only while the cap isn't reached
- 6 Then: escrow → attendee, same transaction

No person approves anything. `contracts/on_sight/lib.rs`

DEMO

Live on devnet: join, walk up, get paid.

1

Join on the event page

Wallet, one selfie, one signature. No transaction.

2

A phone becomes the camera

Scan the QR on the stage screen.

3

Walk up to the camera

Oracles recognise you and report every 2 s.

4

Get paid

The stage shows the payout with an Explorer link.

TRY IT

demo-event.onsight.site

BACKUP

[70 s video walkthrough](#)

WHO CAN DO WHAT

No admin. Nobody can touch the budget.

INSTRUCTION	WHO	WHEN
<code>create_event</code>	anyone, who funds it	any time
<code>report_sighting</code>	an event's oracle + attendee's signed join	during the event
<code>withdraw_remaining</code>	that event's organizer	before start / after end
<code>close_attendance</code>	the oracle that paid its deposit	after the end

Organizer disappears?

Payouts keep working. The rest waits in escrow.

Oracles stop?

Nobody is paid. The organizer withdraws it all after the end.

Until we set the upgrade authority to final, the deployer can still upgrade the code.

WHY BLOCKCHAIN

A database needs trust. A chain doesn't.

✓ **Rules nobody can change**

Not the organizer, not us.

✓ **Paid without asking**

The payout runs by itself.

✓ **Auditable by the sponsor**

Every payout is public.

✓ **Oracles keep each other honest**

Only if no one of them owns the decision.

0.01 SOL

reward

costs

~0.000001 SOL

in fees

Small per-person rewards finally make sense.

ROADMAP

What we'd build next.

1 Staked oracles from an open pool, slashed for disagreeing

2 Liveness detection

3 Several independent oracles per event by default

4 Freeze the program: final upgrade authority